



UPN, PASIÓN POR TRANSFORMAR VIDAS

UNIDAD 2: ÁRBOLES Y GRAFOS

SESIÓN 9: ÁRBOLES BINARIOS

Recorrido inorden, postorden, preorden.

Arboles binarios de Búsqueda ABB.

Dr. Eric Gustavo Coronel Castillo
eric.coronel@upn.pe



DESAFÍO DEL DÍA

"Cada línea de código es un paso hacia una gran solución; las estructuras de datos guían el camino hacia el éxito"



IDEA FUERZA

Programar con intención y estructura
asegura progresos continuos y sólidos.

LOGRO DE LA UNIDAD 2



Al finalizar la unidad, el estudiante implementa algoritmos utilizando árboles y grafos, como resultado del análisis de casos utilizando el lenguaje C# con entorno gráfico, demostrando lógica y habilidad en la implementación de los algoritmos para su proyecto final..



Al término de la sesión, el estudiante aprende algoritmos con árboles, árboles binarios y diversas aplicaciones, usándolos con coherencia.

- Recorrido inorden, postorden, preorden.
- Árboles binarios de Búsqueda ABB.

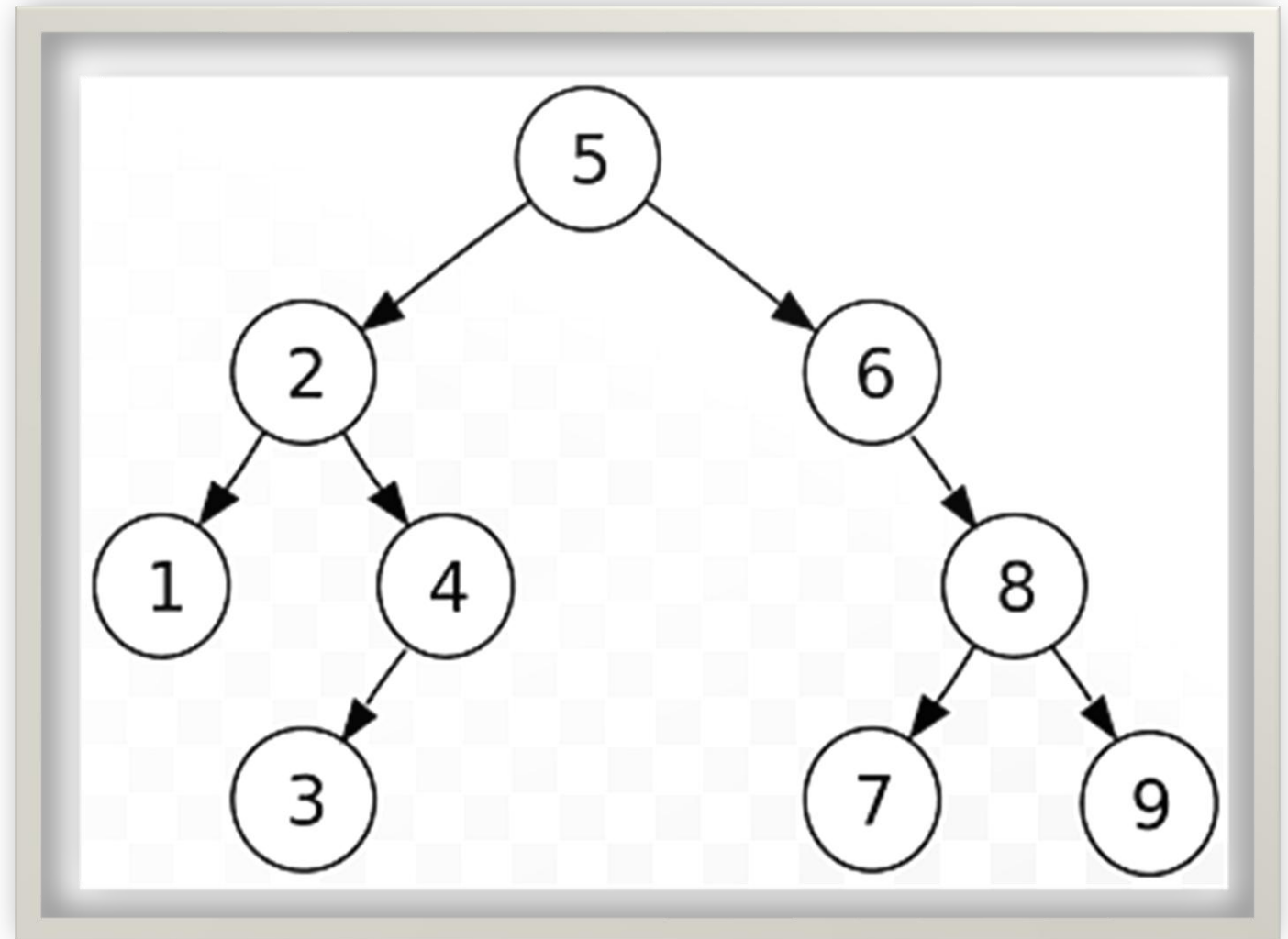


1. ¿Qué es árbol?
2. ¿Qué es un árbol binario?

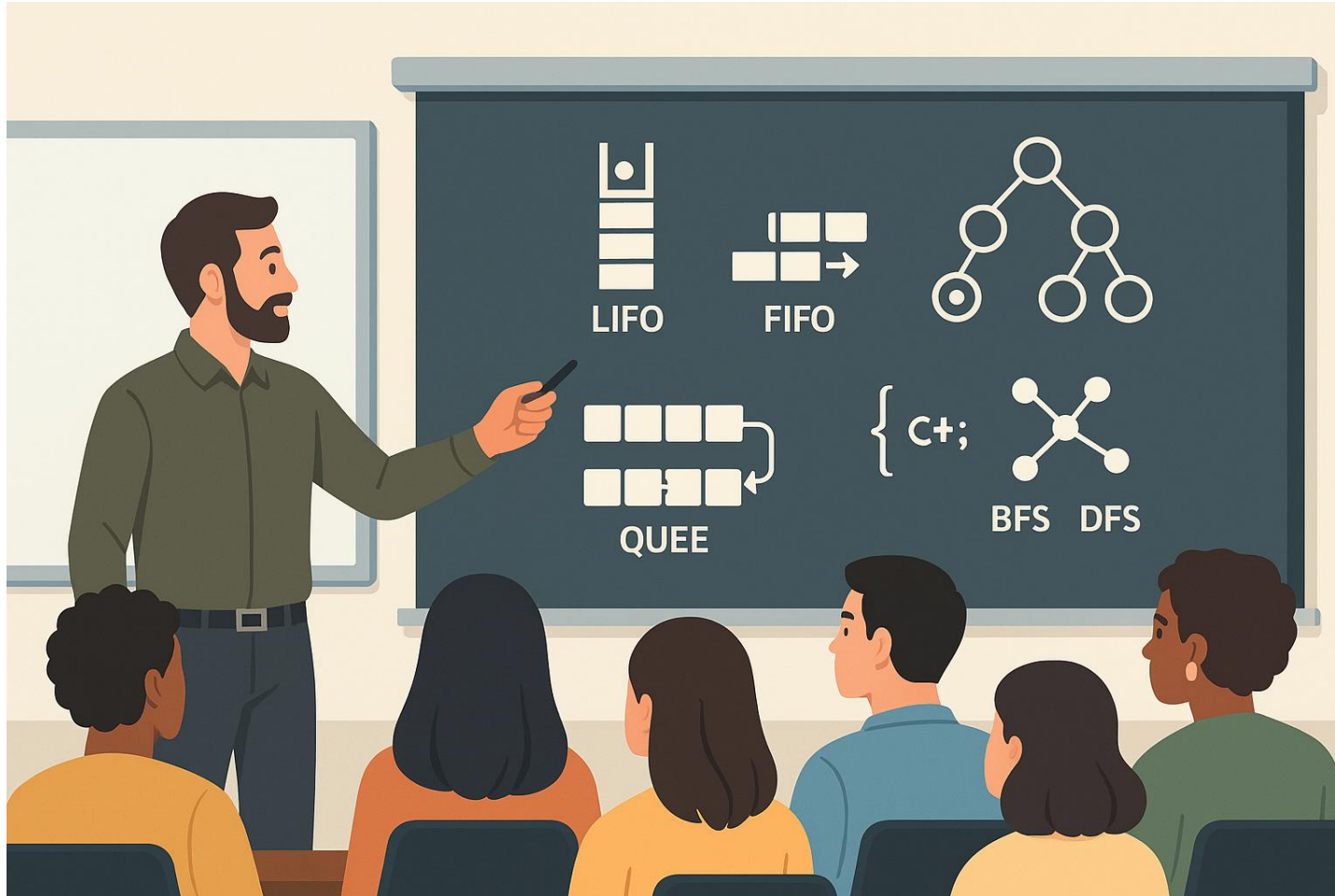


Responder:

1. ¿Es un árbol binario?
2. ¿Cuál es el nodo raíz?
3. ¿Cuáles son los nodos hojas?
4. ¿Cuáles son los nodos rama?
5. ¿Cuántos niveles tiene el árbol?
6. ¿Cumple con los criterios de árbol binario de búsqueda?



DESARROLLO



DATOS IMPORTANTES DE LOS ÁRBOLES



Nodos: Se le llama Nodo a cada elemento que contiene un Árbol.

Nodo Raíz: Se refiere al primer nodo de un Árbol, Solo un nodo del Árbol puede ser la Raíz.

Nodo Padre: Se utiliza este termino para llamar a todos aquellos nodos que tiene al menos un hijo.

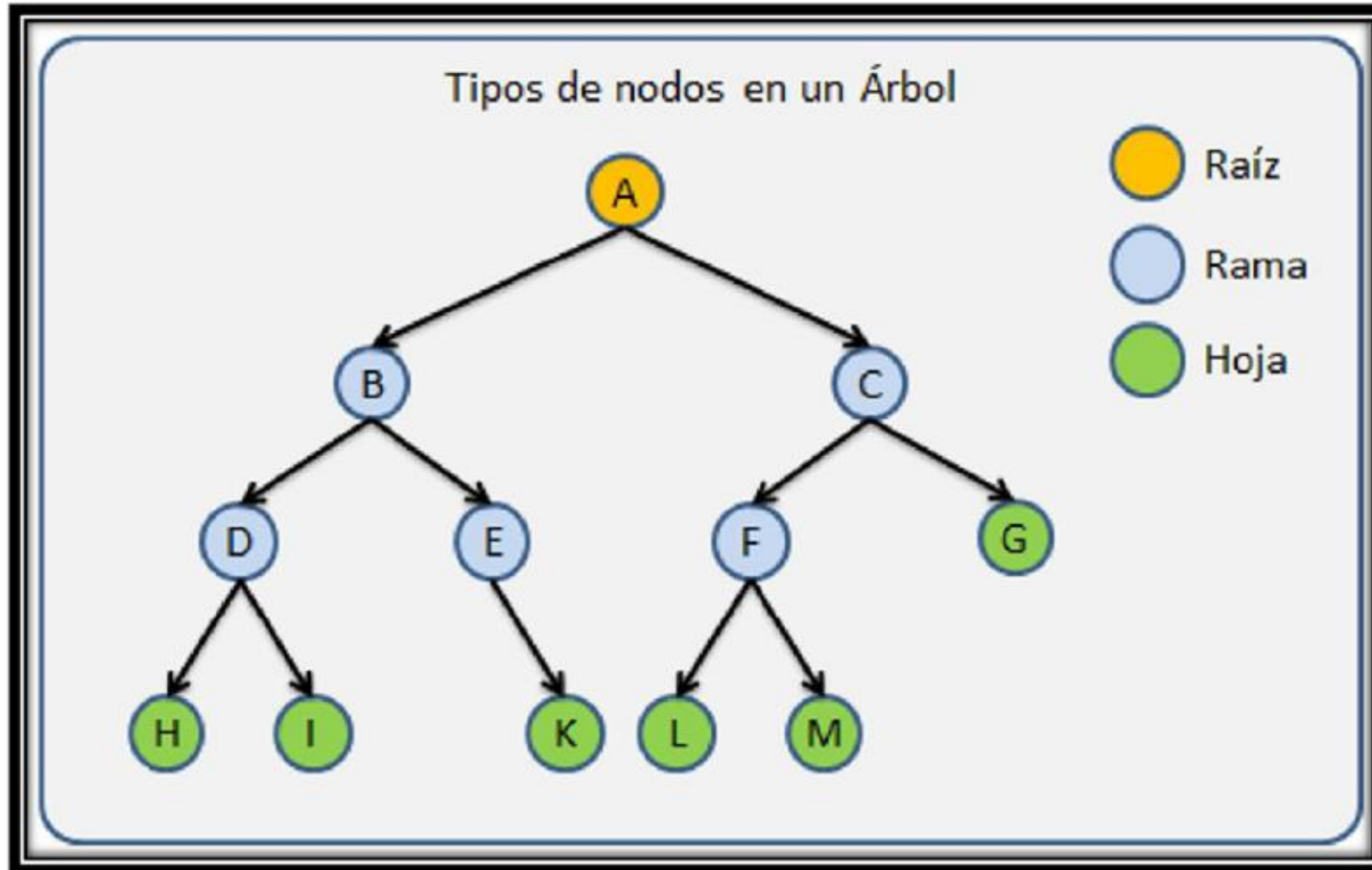
Nodo Hijo: Los hijos son todos aquellos nodos que tiene un padre.

Nodo Hermano: Los nodos hermanos son aquellos nodos que comparte a un mismo padre en común dentro de la estructura.

Nodo Hoja: Son todos aquellos nodos que no tienen hijos, los cuales siempre se encuentran en los extremos de la estructura.

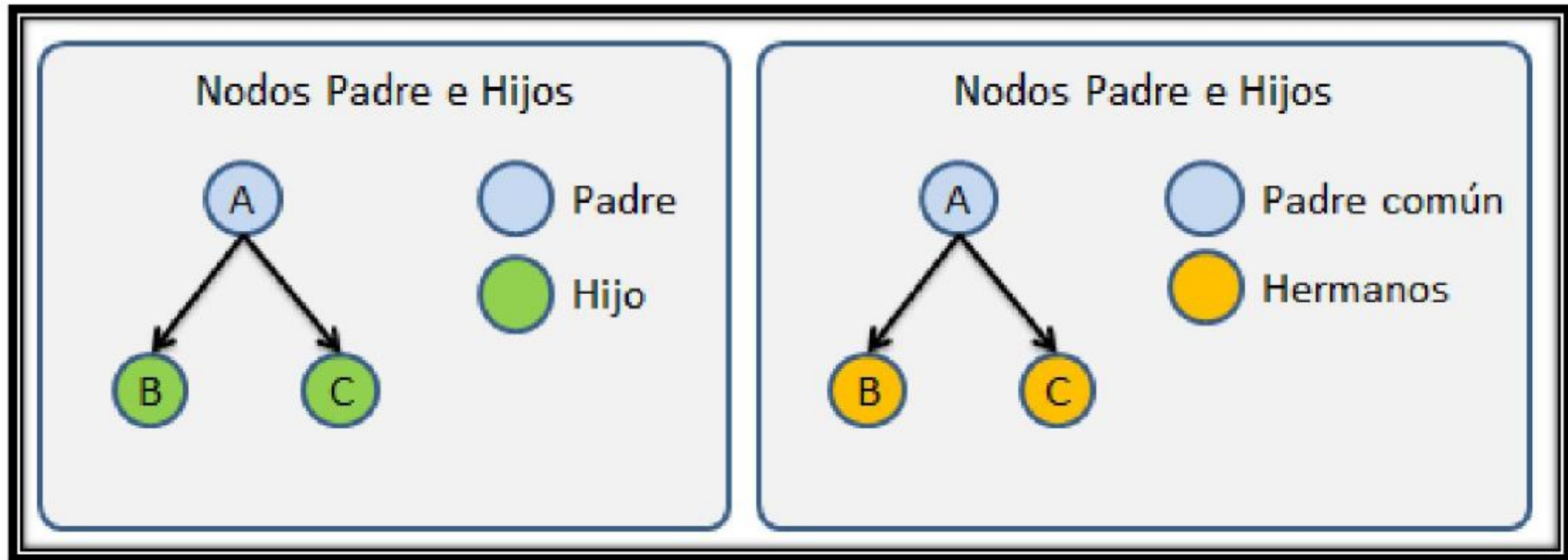
Nodo Rama: Estos son todos aquellos nodos que no son la raíz y que además tiene al menos un hijo.

DATOS IMPORTANTES DE LOS ÁRBOLES



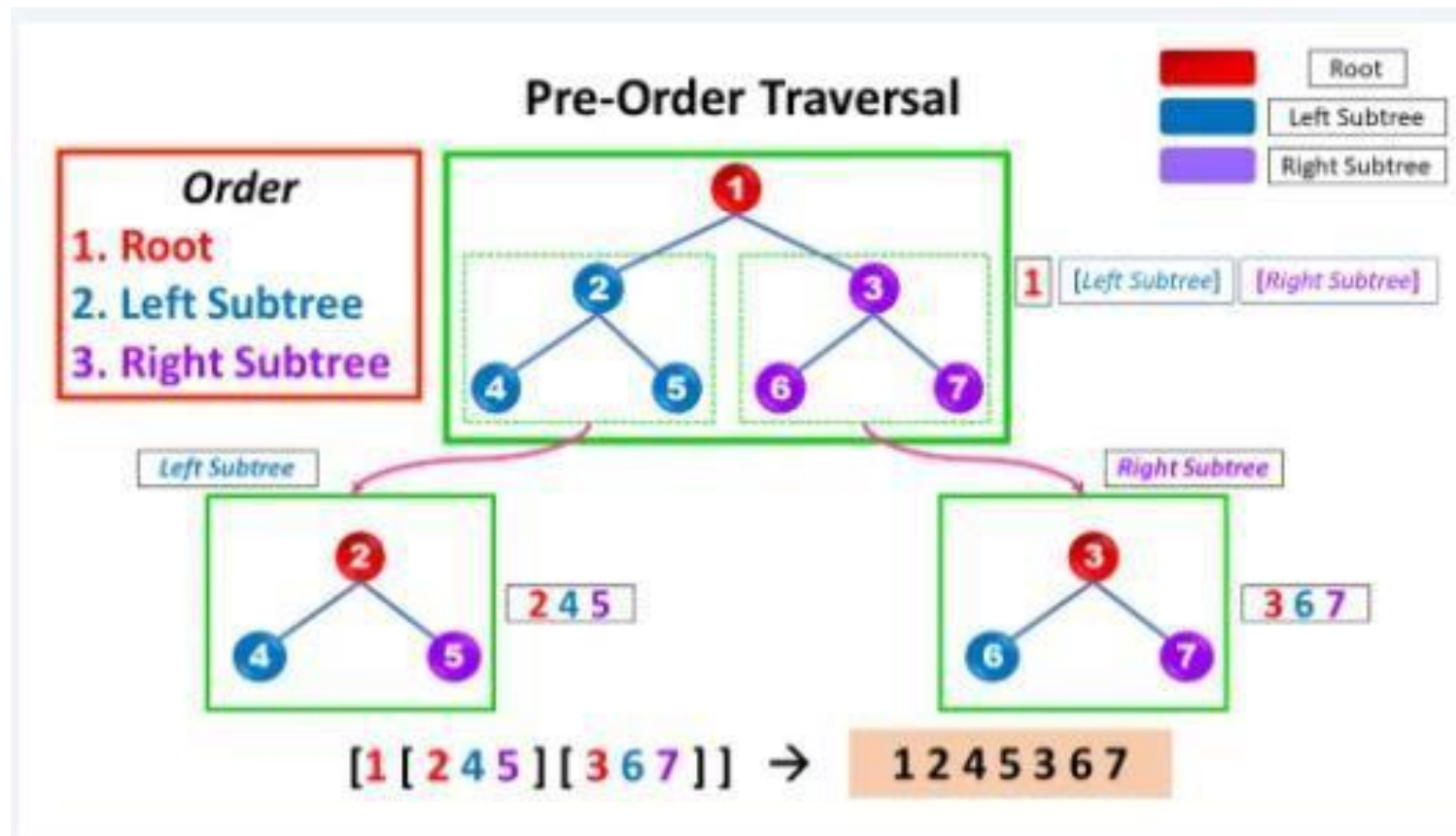
La imagen muestra de forma gráfica cuales son los nodos Raíz, Rama y Hoja.

DATOS IMPORTANTES DE LOS ÁRBOLES



La siguiente imagen muestra de forma gráfica los nodos Padre, Hijo y Hermanos.

En el recorrido PreOrder, se visita primero el nodo raíz, luego el subárbol izquierdo, y finalmente el subárbol derecho.



RECORRIDOS DE UN ÁRBOL BINARIO



PreOrden

Raíz – Izquierda - Derecha

- Visitar nodo actual
- Recorrer subárbol izquierdo
- Recorrer subárbol derecho

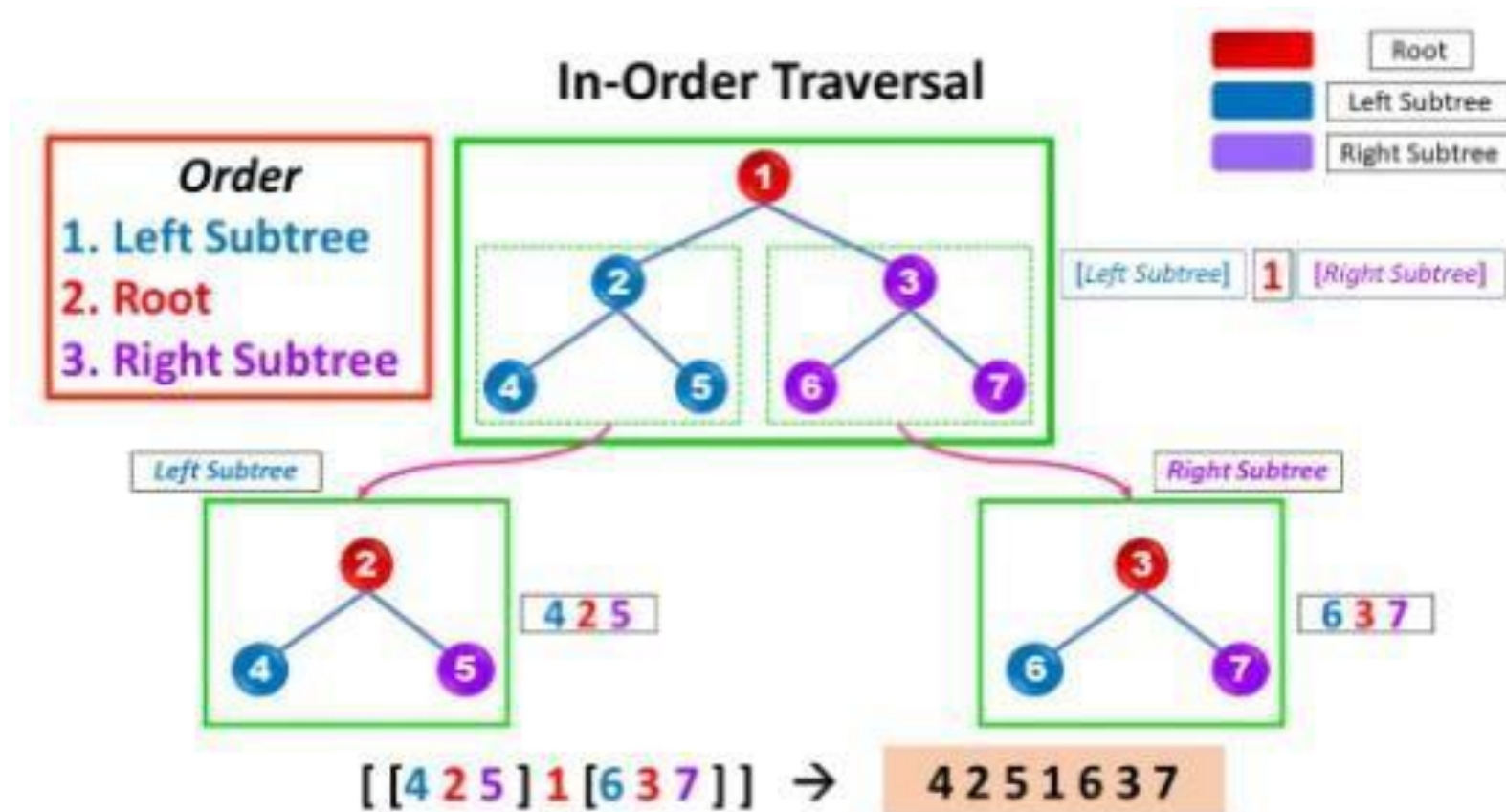
```
public void PreOrden(Nodo nodo)
{
    if (nodo == null)
        return;

    Console.Write(nodo.Valor + " ");
    PreOrden(nodo.Izquierda);
    PreOrden(nodo.Derecha);
}
```

RECORRIDOS DE UN ÁRBOL BINARIO



En el recorrido InOrder, primero se visita el subárbol izquierdo, luego el nodo raíz y finalmente el subárbol derecho.



RECORRIDOS DE UN ÁRBOL BINARIO



InOrden

Izquierda - Raíz - Derecha

- Recorrer subárbol izquierdo
- Visitar nodo actual
- Recorrer subárbol derecho

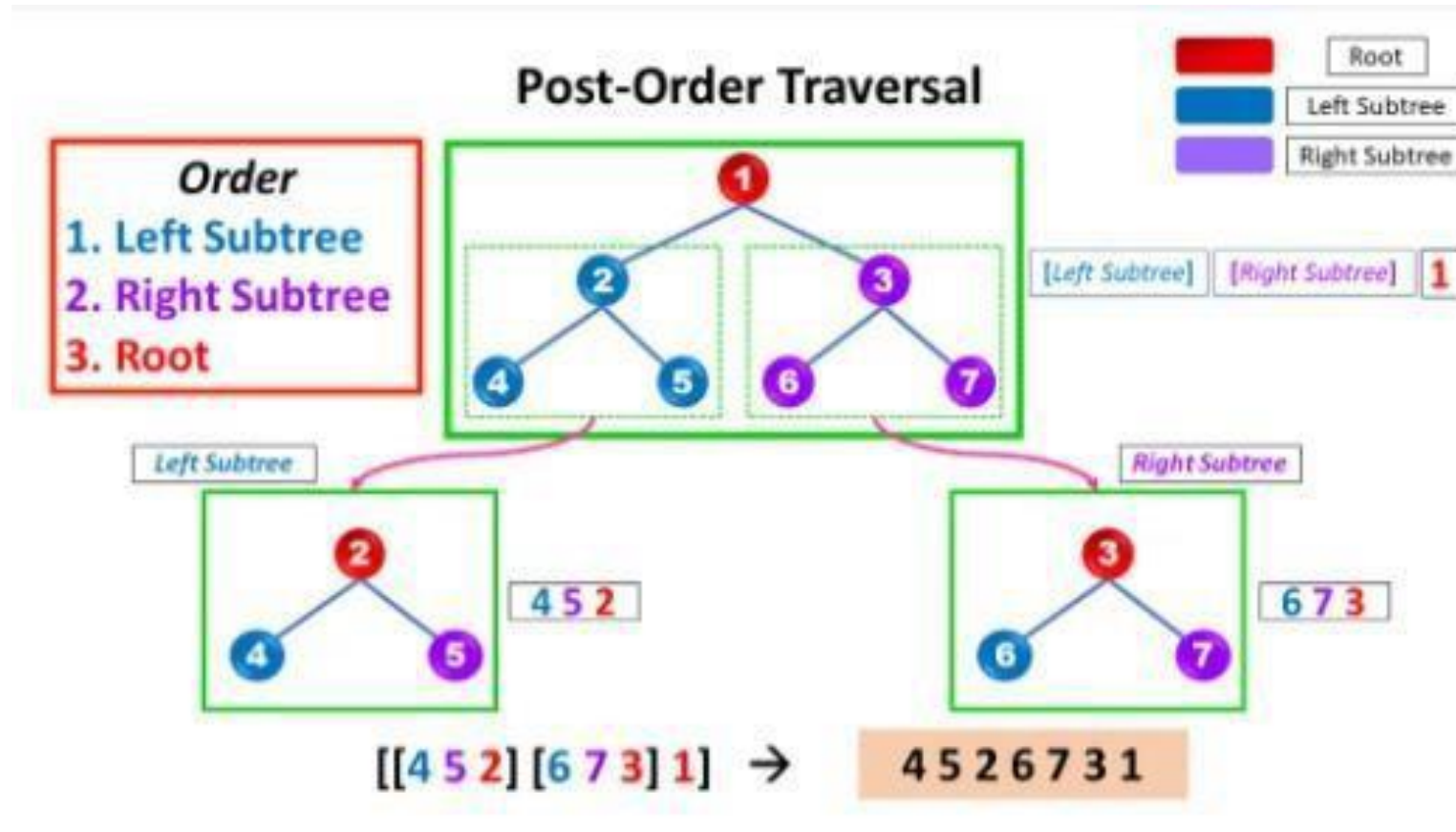
```
public void InOrden(Nodo nodo)
{
    if (nodo == null)
        return;

    InOrden(nodo.Izquierda);
    Console.Write(nodo.Valor + " ");
    InOrden(nodo.Derecha);
}
```

RECORRIDOS DE UN ÁRBOL BINARIO



En el recorrido PostOrder, se visita primero el subárbol izquierdo, luego el subárbol derecho y finalmente el nodo raíz.



RECORRIDOS DE UN ÁRBOL BINARIO



PostOrden

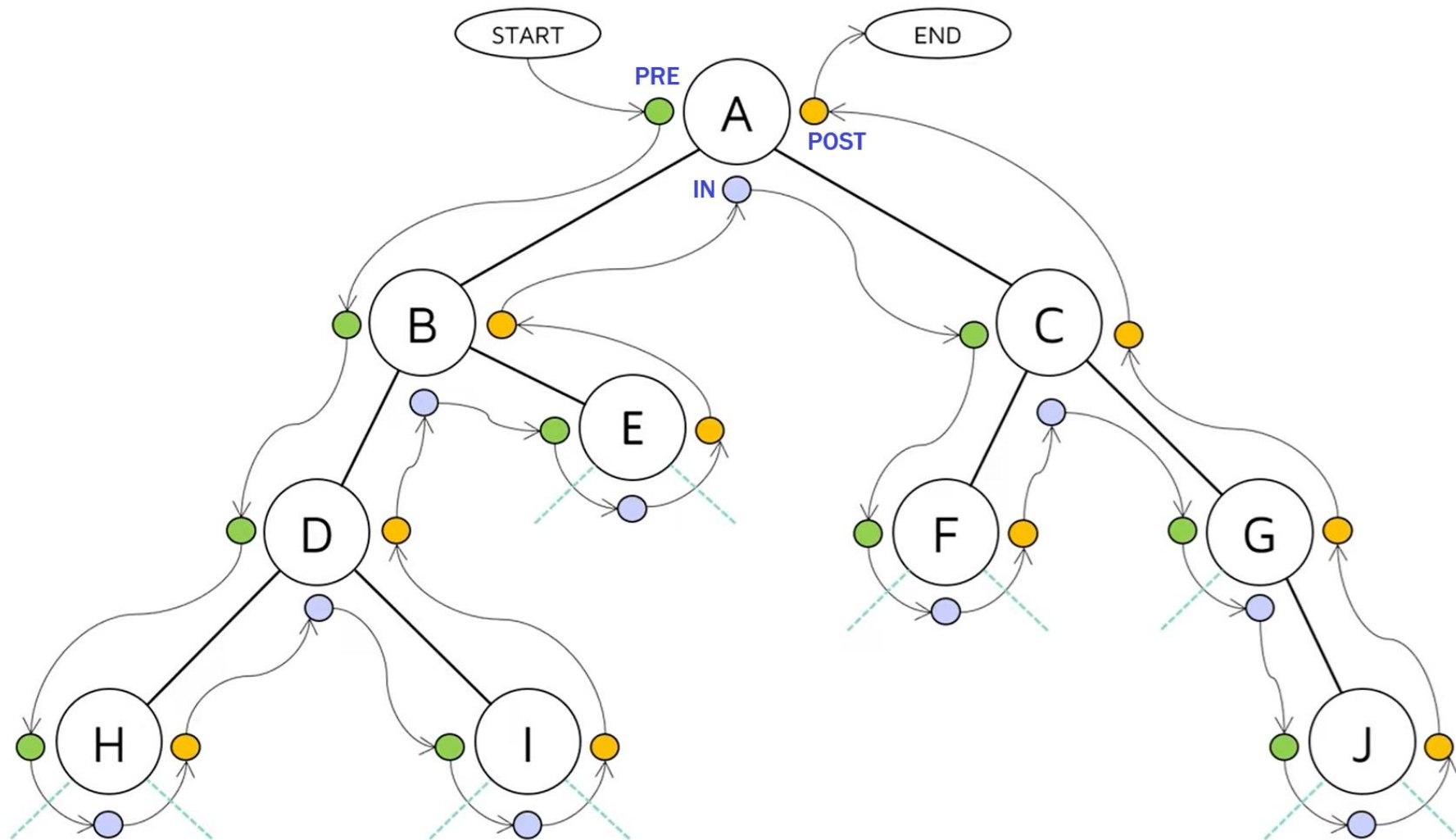
Izquierda - Derecha - Raíz

- Recorrer subárbol izquierdo
- Recorrer subárbol derecho
- Visitar nodo actual

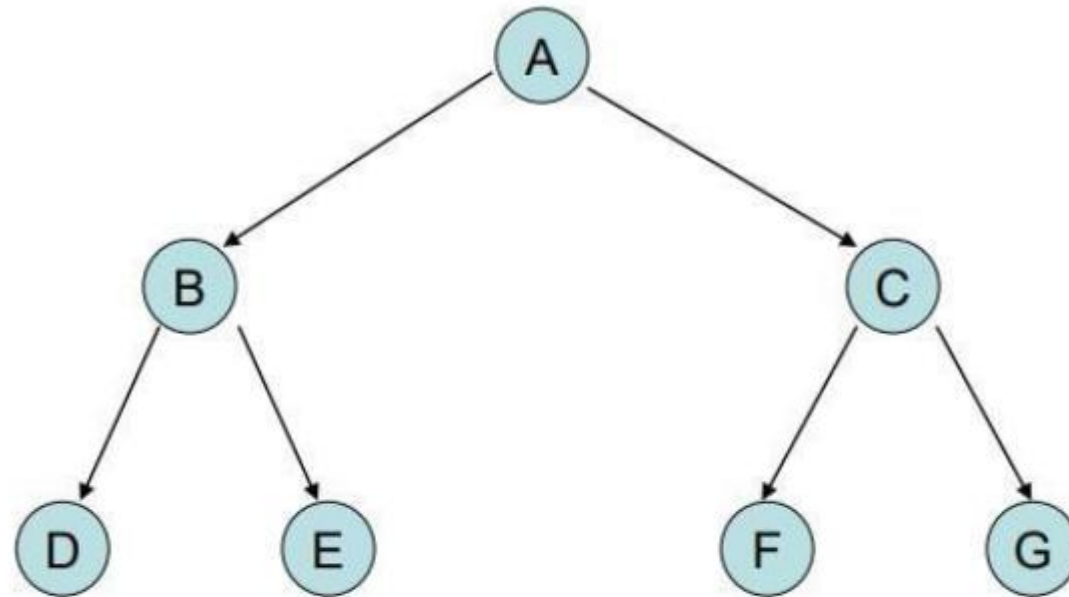
```
public void PostOrden(Nodo nodo)
{
    if (nodo == null)
        return;

    PostOrden(nodo.Izquierda);
    PostOrden(nodo.Derecha);
    Console.Write(nodo.Valor + " ");
}
```

RECORRIDOS DE UN ÁRBOL BINARIO



RECORRIDOS DE UN ÁRBOL BINARIO



Preorden: A, B, D, E, C, F, G

Inorden: D, B, E, A, F, C, G

Postorden: D, E, B, F, G, C, A

¿QUÉ ES UN ABB?

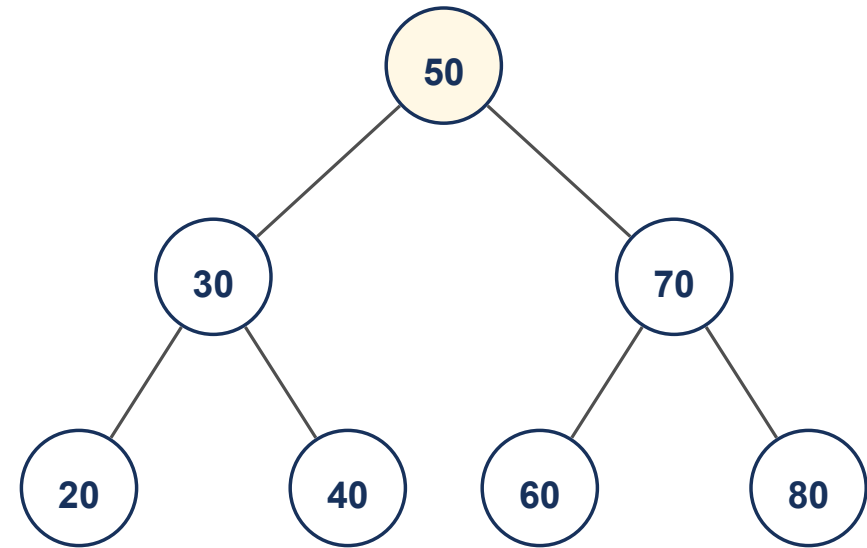


Definición práctica

Un Árbol Binario de Búsqueda es un árbol binario organizado para facilitar operaciones de búsqueda, inserción y eliminación.

Regla de orden:

- Subárbol izquierdo: valores menores
- Subárbol derecho: valores mayores

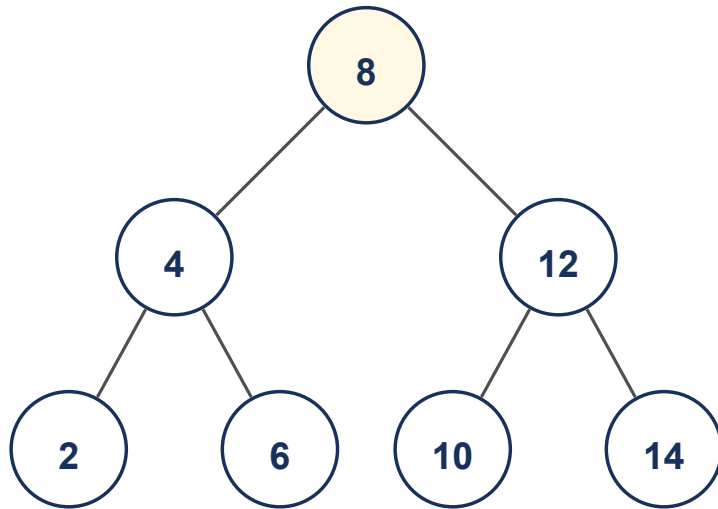


Menores ← raíz → Mayores

ABB VÁLIDO VS. NO VÁLIDO

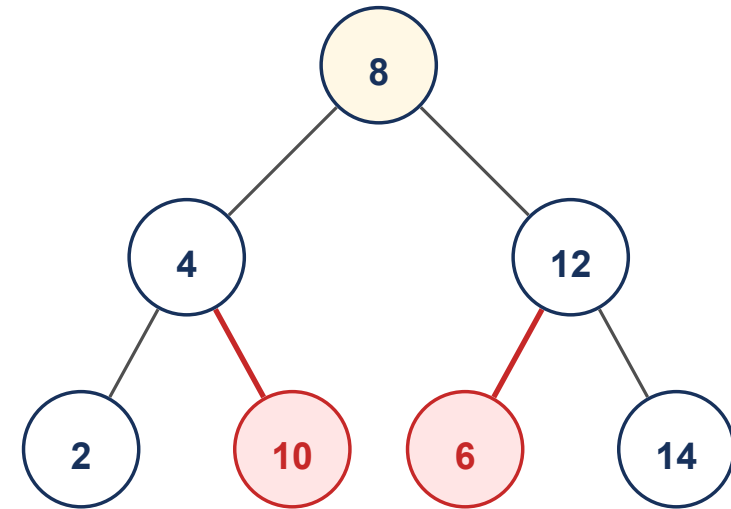


Árbol válido



Todo nodo cumple:
 $\text{izquierda} < \text{nodo} < \text{derecha}$

Árbol no válido



Hay valores ubicados en
el lado incorrecto.

INSERCIÓN EN UN ABB

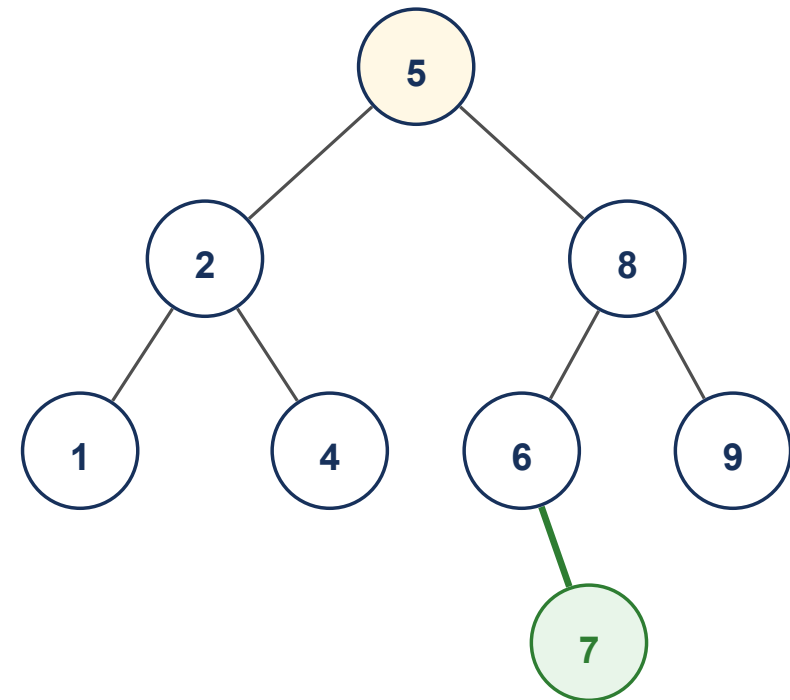


Idea del algoritmo

Para insertar un valor se compara desde la raíz hasta encontrar una posición vacía.

1. Si el árbol está vacío, el nuevo nodo será la raíz.
2. Si el valor es menor, avanzar a la izquierda.
3. Si el valor es mayor, avanzar a la derecha.
4. Repetir hasta insertar el nodo.

Ejemplo: insertar 7



Ruta de comparación:

5 → 8 → 6 → insertar a la derecha.

BÚSQUEDA EN UN ABB

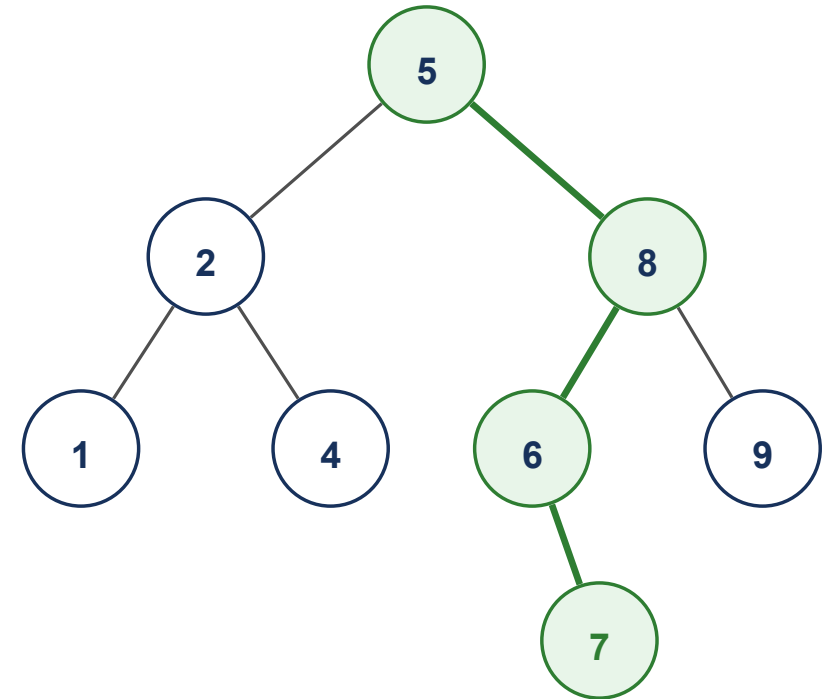


Buscar un valor

La búsqueda aprovecha la regla de orden. En cada comparación se descarta una parte del árbol.

Caso: buscar 7

- $7 > 5$: ir a la derecha
- $7 < 8$: ir a la izquierda
- $7 > 6$: ir a la derecha
- $7 == 7$: encontrado



Solo se recorre una ruta, no todo el árbol.

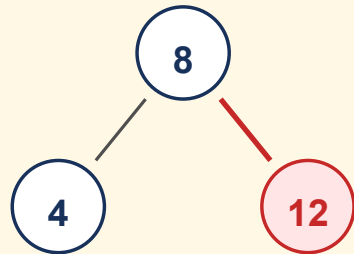
ELIMINACIÓN EN UN ABB



Tres casos principales

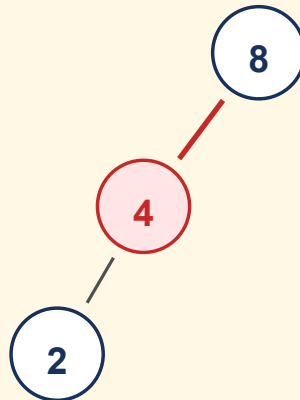
Caso 1: nodo hoja

Se elimina directamente.



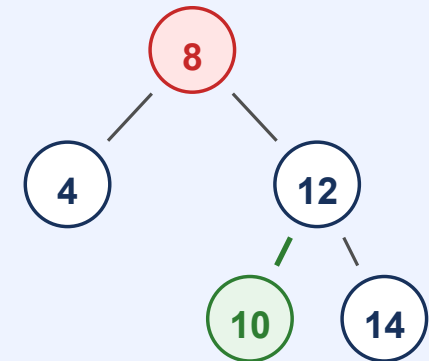
Caso 2: un hijo

El hijo ocupa el lugar del nodo eliminado.



Caso 3: dos hijos

Se reemplaza por el sucesor inorden o predecesor inorden.



En una primera sesión puede trabajarse eliminación de forma conceptual y dejar la implementación completa como extensión.

IMPLEMENTACIÓN BÁSICA EN C#



Clase Nodo e inserción

```
public class Nodo
{
    public int Valor { get; set; }
    public Nodo Izquierda { get; set; }
    public Nodo Derecha { get; set; }

    public Nodo(int valor)
    {
        Valor = valor;
        this.Izquierda = null;
        this.Derecha = null;
    }
}
```

```
private Nodo InsertarRecursoivo
(Nodo nodo, int valor)
{
    if (nodo == null)
        return new Nodo(valor);
    if (valor < nodo.Valor)
        nodo.Izquierda =
            InsertarRecursoivo(nodo.Izquierda, valor);
    else if (valor > nodo.Valor)
        nodo.Derecha =
            InsertarRecursoivo(nodo.Derecha, valor);
    return nodo;
}
```



Método Buscar

```
public bool Buscar(Nodo nodo, int valor)
{
    if (nodo == null)
        return false;

    if (valor == nodo.Valor)
        return true;

    if (valor < nodo.Valor)
        return Buscar(nodo.Izquierda, valor);

    return Buscar(nodo.Derecha, valor);
}
```

Lectura del código

La búsqueda termina cuando:

- El nodo es nulo: el valor no existe.
- El valor coincide con el nodo actual.
- El valor es menor: se busca a la izquierda.
- El valor es mayor: se busca a la derecha.

Este método no recorre todo el árbol: sigue una ruta de comparación.

COMPLEJIDAD Y BUENAS PRÁCTICAS



Rendimiento esperado

Operación	Árbol balanceado	Árbol degenerado
Búsqueda	$O(\log n)$	$O(n)$
Inserción	$O(\log n)$	$O(n)$
Eliminación	$O(\log n)$	$O(n)$

Advertencia didáctica

Un ABB no se balancea solo. Si se insertan datos ordenados, puede parecerse a una lista enlazada y perder eficiencia.

Idea clave: el recorrido inorden de un ABB produce los valores ordenados. Esta propiedad es útil para reportes, validaciones y recorridos ordenados.

APLICACIÓN: EVALUACION DE EXPRESIONES

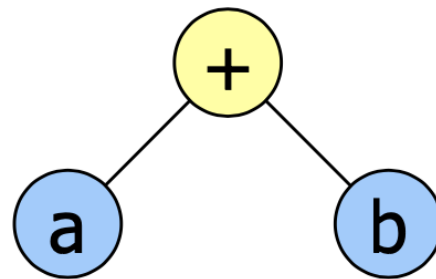


- Ya sabemos lo de las expresiones, cierto?
 - InFija, operador en medio
 - PreFija, operador antes de dos operandos
 - PosFija, operador luego de dos operandos
- Para evaluar una expresion dada, podriamos
 - Pasarla a posfija y usar solo pilas
 - Pasarla a posfija y usar pilas y un arbol de expresion

ARBOL DE EXPRESION



- Árboles que representan expresiones en memoria
 - Todos los operadores tienen dos operandos
 - La raíz puede contener el operador
 - Hijo izq: operando 1, Hijo derecho: operando 2
 - Ejemplo: $(a+b)$



EVALUAR UNA EXPRESION ARITMETICA EN INFIJA

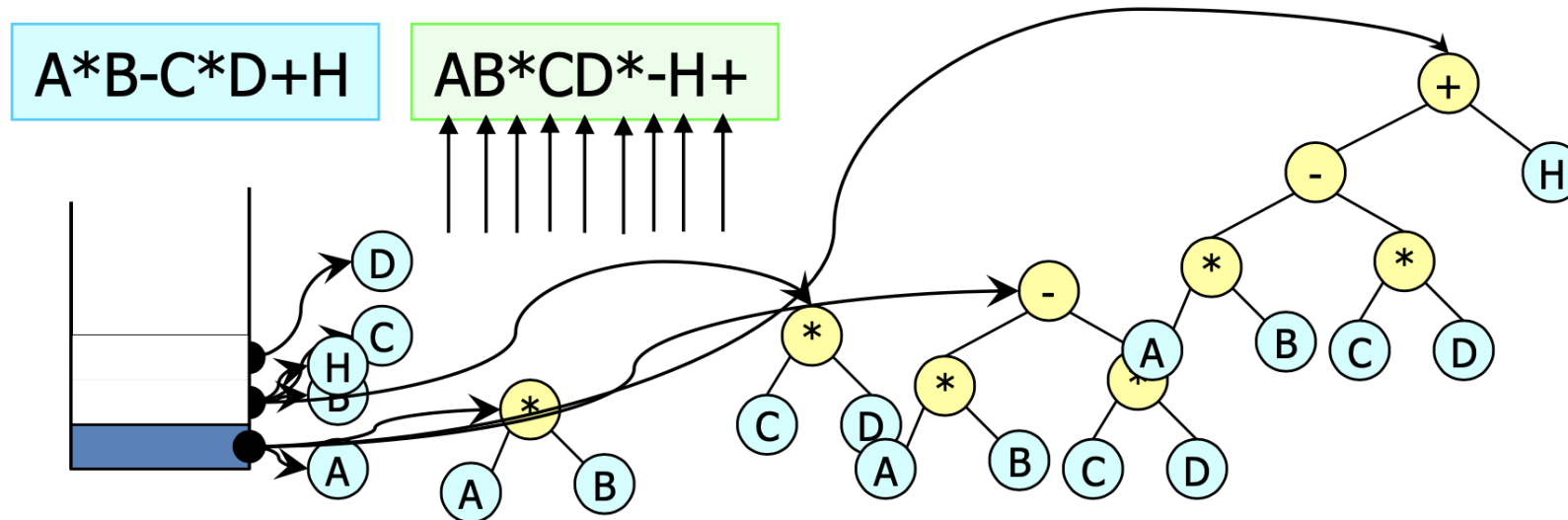


- La expresion se transforma a la expresion posfija
 - Esto, ya sabemos como hacer
- Crear un arbol de expresion
 - Para esto se va a usar una pila y un arbol de caracteres
- Usando el arbol, evaluar la expresion

CREAR UN ARBOL DE EXPRESION



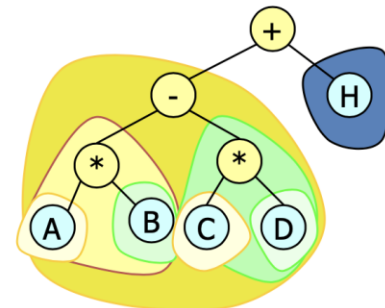
- Los operandos serán siempre nodos hoja del árbol
 - Al revisar un operando, creo una nueva hoja y la recuerdo
- Los operadores serán nodos padre
 - Al revisar un operador, recuerdo las dos últimas hojas creadas y uno todo
 - No debo olvidar el nuevo arbolito que he creado



EVALUACION DE LA EXP. POSTFIJA



- Lo ideal es recuperar los dos operandos, el operador, y ejecutar la opción
 - Que recorrido es el ideal?
 - PostOrden



$(A * B) - (C * D) + H$

Para evaluar el árbol:

Si el árbol tiene un solo nodo y este almacena un operando

El resultado de la evaluación es el valor de ese operando

Si no

1. Res1 = Evaluación subárbol izquierdo
2. Res2 = Evaluación subárbol derecho
3. Recupero la info de la raíz y efectúo la operación allí indicada, entre Res1 y Res2

EJERCICIO EN CLASE



Construya el árbol de expresiones para:

$$a + b * c$$

Construya árbol de expresiones para:

$$(a + b) * c$$

EJERCICIO EN CLASE



Construya árbol de expresión para:

$5 * 4 / 5 + 3 / 2 * 6 - 5 * 9$

EJERCICIO EN CLASE



Construya árbol de expresiones para:

$$(a + b * c) + ((d * e + h) * g)$$

ACTIVIDAD EN CLASE



PROBLEMA 1



Crear un menú con Arboles Binarios que permita generar el registro nuevo de nodo con recorrido de pre-orden, post-orden e InOrden y generar una búsqueda del nodo ingresado.

```
Console.WriteLine("| -----|");
Console.WriteLine("|           Arboles Binarios|");
Console.WriteLine("| -----|");
Console.WriteLine("| 1. Registrar un nuevo nodo|");
Console.WriteLine("| 2. Recorrido en pre-orden|");
Console.WriteLine("| 3. Recorrido en post-orden|");
Console.WriteLine("| 4. Recorrido en In-orden|");
Console.WriteLine("| 5. Buscar nodo|");
Console.WriteLine("| -----|");
Console.WriteLine(" Escoja una opción: ");
```

PROBLEMA 2



Problema propuesto

Crear un menú con Árbol Binario de Búsqueda que permita insertar nodos, buscar un valor y mostrar los recorridos del árbol.

Opciones mínimas del menú

1. Insertar nodo
2. Buscar nodo
3. Recorrido preorden
4. Recorrido inorden
5. Recorrido postorden
6. Salir

Evidencia esperada

- Código funcional en C#.
- Prueba con al menos 8 valores.
- Captura o demostración de cada recorrido.
- Validación de búsqueda existente y no existente.

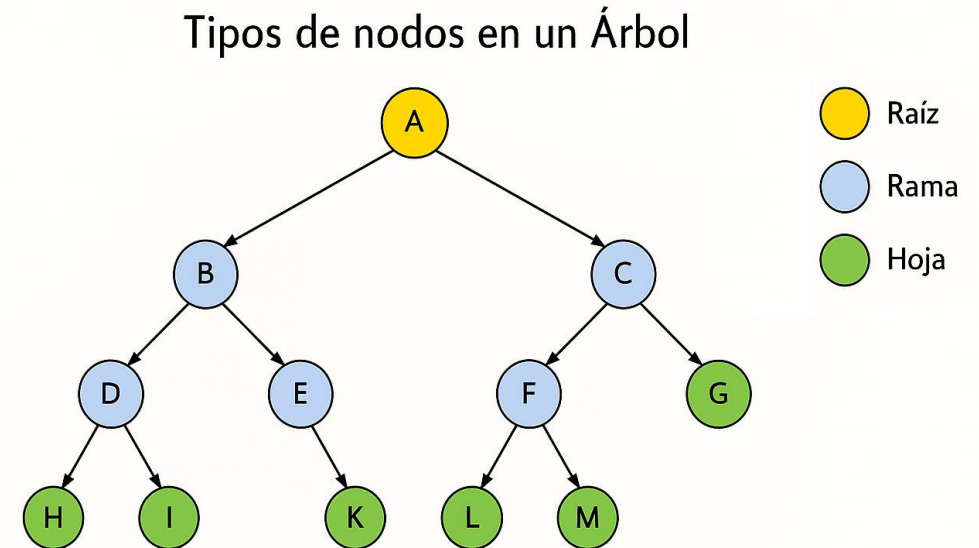
Recomendación: usar el árbol de la página 7 para una prueba inicial.

CONCLUSIONES



Los árboles binarios permiten organizar datos de manera jerárquica. Sus recorridos preorden, inorden y postorden permiten procesar los nodos en diferentes secuencias.

En el caso del Árbol Binario de Búsqueda, la ubicación ordenada de los valores facilita operaciones como inserción y búsqueda.



¿PREGUNTAS O COMENTARIOS?



COMPROBACIÓN DEL LOGRO





Cuestionario en la plataforma

Nivel	Rango
Nivel 4	17 – 20
Nivel 3	13 – 16
Nivel 2	9 – 12
Nivel 1	0 - 8

Comprobación del Logro

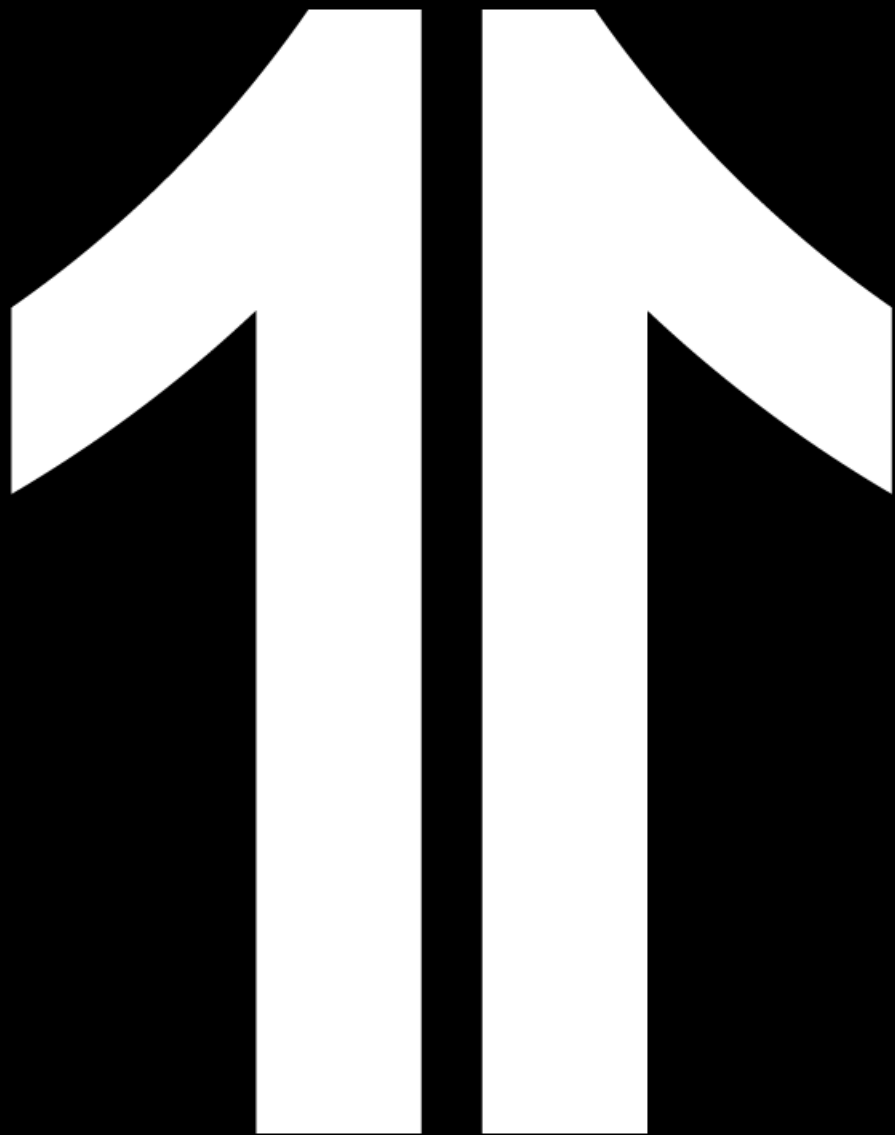


Criterio	Nivel 4 (Excelente)	Nivel 3 (Bueno)	Nivel 2 (Regular)	Nivel 1 (Deficiente)
Comprensión conceptual	Demuestra comprensión completa de los conceptos clave.	Comprende la mayoría de los conceptos clave.	Muestra comprensión parcial, con algunas confusiones.	No demuestra comprensión clara de los conceptos.
Aplicación de conocimientos	Aplica correctamente los conceptos en ejemplos o escenarios.	Aplica conceptos con algunos errores menores.	Aplica de forma superficial o con errores evidentes.	No logra aplicar los conceptos o lo hace incorrectamente.
Claridad y precisión en respuestas	Las respuestas son claras, bien estructuradas y precisas.	Las respuestas son mayormente claras, con leves imprecisiones.	Respuestas poco claras o con ideas mal organizadas.	Respuestas confusas, incompletas o incoherentes.
Cobertura del contenido esperado	Responde completamente las 5 preguntas, desarrollando bien cada una.	Responde correctamente 4 de las 5 preguntas.	Responde correctamente 2 o 3 preguntas.	Responde solo 1 pregunta correctamente o no responde.

BIBLIOGRAFIA REFERENCIAL



- Ceballos Sierra, F. Microsoft C#: Curso de Programación (2a.ed.) 2014
<https://elibronet.eu1.proxy.openathens.net/es/lc/upnorte/titulos/106417>
- Cesar Liza Avila; Estructura de datos con C/C++



GRACIAS

